

IE7615 Deep Learning for AI

Discriminative Project Milestone 1

Model Comparison for Image Classification

Group – 10

Kush Pulindra Patel – 002560878

Varun Ponugoti – 002533962

Sathya Sai Rajan – 002507375

Vignesh Radhakrishnan Nambiar – 002598467

Submission date: 10/04/2026

Model Comparison for Object Image Classification

Kush Pulindra Patel
MS Data Analytics
Northeastern University

Varun Ponugoti
MS Robotics
Northeastern University

Sathya Sai Rajan
MS Data Analytics
Northeastern University

Vignesh Radhakrishnan Nambiar
MS Data Analytics
Northeastern University

Abstract—We present an empirical comparison of four deep convolutional models for classifying photographs of 73 distinct objects: a four-block convolutional neural network (CNN) trained from scratch and three ImageNet-pretrained architectures adapted by transfer learning: MobileNetV2 [3], ResNet50 [2], and EfficientNetB0 [4]. The dataset consists of roughly 100 images per class. Before any training, we validate every file and remove exact duplicates using MD5 hashing so that no image can appear in both the training and test sets. All four models share the same stratified 70/15/15 data split, the same augmentation pipeline, the same loss, and the same callbacks, and the three pretrained models use an identical two-stage schedule of frozen-backbone head training followed by fine-tuning of the last 30 backbone layers. On a held-out test set of 1,098 images, EfficientNetB0 achieves the best accuracy of 94.17% and a macro F1-score of 0.941, narrowly ahead of ResNet50 (93.81%) while using about one-sixth of its parameters. The scratch-trained baseline reaches only 74.50%, a gap of almost 20 percentage points.

Index Terms—image classification, transfer learning, convolutional neural networks, EfficientNet, fine-tuning

I. INTRODUCTION

Object recognition is a core capability for many practical systems. In different settings, a model must reliably distinguish a large number of object categories in which some may look very similar from ordinary photographs.

Convolutional neural networks learn a hierarchy of visual features, from edges and textures up to object parts and whole objects directly from pixels. When enough data is available, this removes the need for hand-crafted features. With only about 100 images per class, however, a network trained from scratch has little data from which to learn general-purpose features. Transfer learning addresses this by reusing a network pretrained on a large dataset such as ImageNet [1]; features learned in the early and middle layers of such networks are known to transfer well to new visual tasks.

We compare four models on the same 73-class object dataset: a lightweight CNN trained from scratch as a baseline, and three pretrained architectures that represent different points in the trade-off between accuracy and model size. MobileNetV2 is designed for mobile devices and is very small; ResNet50 is a deep, high-capacity network; and EfficientNetB0 uses compound scaling to obtain high accuracy for a modest parameter count. To make the comparison fair, all models use the same data, preprocessing, augmentation, loss function, and stopping criteria.

This report answers three questions.

- 1) How much does transfer learning from ImageNet improve accuracy over a CNN trained from scratch on a dataset of roughly 100 images per class?
- 2) Among the pretrained architectures, how do accuracy and model size trade off and which model is the most suitable for this dataset?
- 3) Where do the remaining errors of the best model concentrate, and what limits the conclusions that can be drawn from this evaluation?

The remainder of the paper is organized as follows. Section II states the classification problem and the dataset. Section III specifies the data pipeline, the four models, and the training procedure in enough detail to reproduce every experiment. Section IV reports quantitative results. Section V summarizes findings, limitations, and future directions.

II. PROBLEM STATEMENT

We consider supervised multi-class image classification. The dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ consists of RGB images $\mathbf{x}_i \in \mathbb{R}^{224 \times 224 \times 3}$ and labels $y_i \in \{1, \dots, C\}$ with $C = 73$ object classes. The goal is to learn a classifier f_θ that outputs a probability vector $\hat{\mathbf{p}} = f_\theta(\mathbf{x}) \in [0, 1]^C$, with the predicted class $\hat{y} = \arg \max_c \hat{p}_c$.

The parameters θ are learned by minimizing the categorical cross-entropy over the training set $\mathcal{D}_{\text{train}}$:

$$\mathcal{L}(\theta) = -\frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(\mathbf{x}, y) \in \mathcal{D}_{\text{train}}} \log \hat{p}_y(\mathbf{x}; \theta). \quad (1)$$

The raw dataset contains 7,345 files organised into one folder per class (images_OBJ001 to images_OBJ073). All the images are 224×224 pixels in RGB format. The classes are well balanced: after cleaning, each class has between 94 and 113 images. Table I summarizes the dataset.

TABLE I
DATASET SUMMARY

Property	Value
Raw files	7,345
Images after cleaning	7,315
Classes	73
Images per class	94 – 113 (mean 100)
Input size	224×224 RGB
Train / validation / test	5,120 / 1,097 / 1,098

A model is judged by its performance on a held-out test set that is never used for training or early stopping. Because the

dataset is balanced and every class is equally important, we report both accuracy and the macro-averaged F1-score.

III. PROPOSED METHOD

The pipeline has four stages: data validation and de-duplication, stratified splitting with a shared input pipeline, model construction, and two-stage training. All four models pass through the same first two stages.

A. Data Validation and De-duplication

Every file is first checked for a `.jpg` or `.jpeg` extension and then opened with PIL's `verify()` to detect corruption. Two files were excluded because they were not JPEG images; no corrupted files were found.

Next, an MD5 hash is computed for every remaining image, and any image whose hash matches an earlier one is removed, keeping the first copy. This step removed 28 exact duplicates. None of the duplicate pairs crossed class folders so there were no conflicting labels. De-duplication is performed *before* splitting; otherwise, two copies of the same image could be placed in the training and test sets which would inflate test accuracy. All 30 excluded files are written to a log file for traceability.

B. Data Splitting and Input Pipeline

The 7,315 cleaned images are split with stratification by class into 70% training, 15% validation, and 15% test data, using a fixed random seed of 42. Stratification keeps the class proportions equal across the three sets, giving about 15 test images per class.

Images are loaded with a `tf.data` pipeline that decodes each JPEG as a 3-channel RGB image, resizes it to 224×224 , and batches it with a batch size of 32. The training set is shuffled every epoch, and batches are prefetched to keep the GPU busy.

C. Augmentation and Input Normalization

To reduce overfitting on the small dataset, all four models begin with the same augmentation block, which is active only during training: a random horizontal flip, a random rotation of up to $\pm 10\%$ of a full turn, a random zoom of up to $\pm 10\%$, and a random contrast change of up to $\pm 10\%$. Placing the augmentation inside the model guarantees that every architecture sees exactly the same kind of augmented data.

Input normalization follows each architecture's requirements. The baseline CNN rescales pixel values to $[0, 1]$. MobileNetV2 and ResNet50 apply their respective Keras `preprocess_input` functions which reproduce the normalization used during ImageNet pretraining. EfficientNetB0 contains its own normalization layer and therefore receives raw pixel values in $[0, 255]$.

D. Baseline CNN

The baseline is a CNN trained from scratch. It contains four convolutional blocks with 32, 64, 128, and 256 filters. Each block applies a 3×3 convolution with same padding and ReLU activation, batch normalization, and 2×2 max pooling. The blocks are followed by global average pooling, dropout rate of 0.4, and a 73-unit softmax layer. The model has 409,097 parameters in total.

E. Transfer-Learning Models

The three pretrained models share one design. An ImageNet-pretrained backbone is loaded without its original classification layer and with global average pooling producing a feature vector of dimension d . A dropout layer with rate 0.3 and a new 73-unit softmax layer are added on top. The new classification head contains $(d + 1) \times C$ parameters, which gives 93,513 parameters for MobileNetV2 and EfficientNetB0 ($d = 1280$) and 149,577 for ResNet50 ($d = 2048$).

F. Training Procedure

All models are trained with the Adam optimizer on the loss in Eq. (1), using the same two callbacks: early stopping with a patience of 5 epochs on validation accuracy, which restores the best weights, and a learning-rate reduction by a factor of 0.3 after 2 epochs without improvement in validation loss.

The baseline CNN is trained in a single stage for up to 40 epochs with a learning rate of 10^{-3} .

The pretrained models are trained in two stages:

- 1) *Head training*. The backbone is frozen, and only the new classification head is trained for up to 15 epochs with a learning rate of $\eta_1 = 10^{-3}$.
- 2) *Fine-tuning*. The last 30 layers of the backbone are unfrozen, while all batch-normalization layers stay frozen so that their pretrained statistics are not disturbed by small batches. The model is then trained for up to 15 more epochs with a much smaller learning rate, $\eta_2 = 10^{-5} = \eta_1/100$.

G. Evaluation Metrics

For each class c , precision P_c , recall R_c , and F1-score are computed in a one-vs-rest manner from the true positives TP_c , false positives FP_c , and false negatives FN_c :

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c}, \quad (2)$$

$$F1_c = \frac{2P_cR_c}{P_c + R_c}. \quad (3)$$

The macro F1-score averages the per-class scores with equal weight:

$$F1_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C F1_c. \quad (4)$$

Accuracy is the fraction of test images classified correctly. Unlike accuracy, the macro F1-score reveals whether a model performs poorly on particular classes.

IV. RESULTS

All experiments were run on Google Colab with an NVIDIA T4 GPU, using TensorFlow/Keras 2.20.0 with a fixed random seed of 42. Each model was trained once and evaluated once on the same held-out test set of 1,098 images.

A. Model Comparison

Table II reports the test performance of all four models. The epochs column gives the total number of epochs run before early stopping, including both stages for the pretrained models.

TABLE II
TEST SET PERFORMANCE ($n = 1,098$ IMAGES)

Model	Acc.	Macro F1	Params	Epochs
BaselineCNN	0.7450	0.7465	0.41M	28
MobileNetV2	0.9098	0.9102	2.35M	24
ResNet50	0.9381	0.9392	23.74M	23
EfficientNetB0	0.9417	0.9413	4.14M	30

All three pretrained models outperform the baseline by a wide margin, between 16.5 and 19.7 percentage points in accuracy. ResNet50 and EfficientNetB0 are almost tied, separated by 0.36 percentage points, but ResNet50 needs about 5.7 times as many parameters. MobileNetV2, the smallest pretrained model, is about 3 percentage points behind the best model. For every model, accuracy and macro F1 differ by less than 0.002, which is expected given the balanced classes.

B. Training Dynamics

Fig. 1 shows the validation accuracy of each model per epoch. The three pretrained models start at a high validation accuracy of about 0.72–0.84 after the first epoch, because their pretrained features are already informative, and they reach a stable plateau within a few epochs. ResNet50 and EfficientNetB0 level off at about 0.93 and MobileNetV2 at about 0.89–0.90. Fine-tuning produces only small further gains, which suggests that the frozen ImageNet features already separate most of the 73 classes well.

The baseline CNN behaves very differently. It starts near chance level ($1/73 \approx 0.014$), improves slowly, and shows large swings from one epoch to the next before reaching about 0.75 when early stopping ends training. Its validation accuracy was still trending upward at that point, which indicates under-fitting rather than overfitting.

For ResNet50 and EfficientNetB0, the final validation accuracy of about 0.93 closely matches the test accuracy of about 0.94, indicating that the models generalize well to unseen images.

C. Per-Class Analysis of the Best Model

For EfficientNetB0, the macro-averaged precision, recall, and F1-score are all 0.94. Fig. 2 shows its confusion matrix, which is strongly diagonal. Of the 73 classes, 20 are classified perfectly, 37 have an F1-score of at least 0.97, and 57 have an F1-score above 0.90. The remaining errors are spread thinly, usually one or two images per off-diagonal cell.

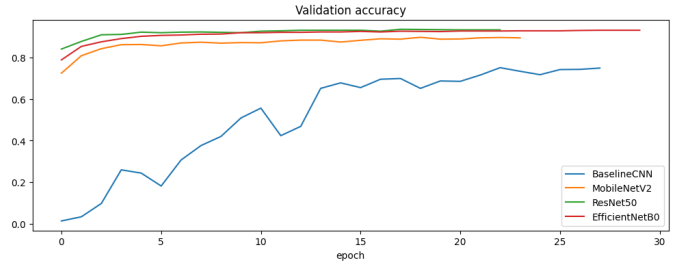


Fig. 1. Validation accuracy per epoch for all four models

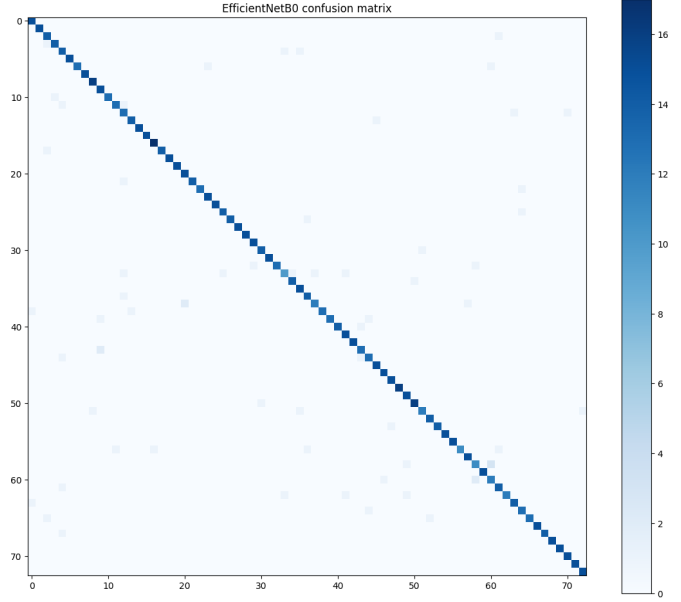


Fig. 2. Confusion matrix of EfficientNetB0 on the test set (rows: true class, columns: predicted class; indices 0–72 correspond to OBJ001–OBJ073).

Table III lists the five classes with the lowest F1-scores. The clearest systematic error is between OBJ059 and OBJ061, which are confused with each other in both directions, suggesting that these two objects are visually similar. OBJ034 has the lowest recall (0.67), with its errors spread over several other classes rather than one. Because each class has only about 15 test images, a single misclassified image changes a class’s recall by about 7 percentage points.

TABLE III
WEAKEST CLASSES FOR EFFICIENTNETB0

Class	Precision	Recall	F1	Support
OBJ034	0.83	0.67	0.74	15
OBJ059	0.79	0.73	0.76	15
OBJ061	0.75	0.80	0.77	15
OBJ013	0.76	0.87	0.81	15
OBJ005	0.78	0.88	0.82	16

D. Model Selection

We select EfficientNetB0 as the final model. It has the highest test accuracy (0.9417) and macro F1-score (0.9413);

its validation and test accuracy agree closely; and with 4.14M parameters it is about 5.7 times smaller than ResNet50, the only model with comparable accuracy, which makes it faster at inference and easier to deploy. If model size were the overriding concern, MobileNetV2 would be a reasonable alternative at a cost of about 3 percentage points of accuracy.

V. CONCLUSION

We compared a CNN trained from scratch with three ImageNet-pretrained models on a 73-class object dataset with about 100 images per class using an identical data pipeline and training setup for all models. Transfer learning was decisively better: the pretrained models reached 91–94% test accuracy, compared with 74.5% for the baseline, and they converged within a few epochs while the baseline was still slowly improving. Among the pretrained models, EfficientNetB0 gave the best balance of accuracy and size, achieving 94.17% accuracy and a macro F1-score of 0.941 with 4.14M parameters. Its few remaining errors are concentrated in a small number of classes, most notably the mutually confused pair OBJ059 and OBJ061.

CODE AVAILABILITY

The code for all four models tested in this study, together with a README that describes how to run it, is available at <https://github.com/Varun-Ponugoti/DLAI.git>.

REFERENCES

- [1] Jia Deng et al. “ImageNet: A large-scale hierarchical image database”. In: *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*. 2009, pp. 248–255.
- [2] Kaiming He et al. “Deep residual learning for image recognition”. In: *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778.
- [3] Mark Sandler et al. “MobileNetV2: Inverted residuals and linear bottlenecks”. In: *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 4510–4520.
- [4] Mingxing Tan and Quoc V. Le. “EfficientNet: Rethinking model scaling for convolutional neural networks”. In: *Proc. Int. Conf. Machine Learning (ICML)*. 2019, pp. 6105–6114.